

HASHDIT

HashDit

Smart Contract Security

ListaDao MarketFactory V3Provider Market Creation

Security Audit Report

ListaDao

19 Oct 2026 – 20 Oct 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	19 Oct 2026 - 20 Oct 2026
Overall Risk	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/255
Commit	feat/marketfactory-v3-markets

Smart Contract List

No.	Contract Name	Verdict	Details
1	MarketFactory.sol	Informational	[I01] [I03]
2	V3MarketLib.sol	Informational	[I02] [I04]
3	IV3Liquidator.sol	-	Green

Findings

[I01] Deposit guard does not guarantee a seeded market and can be bypassed before seeding	
Contract	V3MarketLib.sol
Severity	Informational

Description	**Description**
	During market creation, V3MarketLib enforces require(totalSupply > 0 || depositWhitelistEnabled()). This check only proves the market is "not unsafe" at the instant of creation, not that it is operational, and it is strictly point-in-time. It allows a market to be created while supply == 0 as long as the deposit whitelist is enabled. A market in that state prices at 0 until an off-chain whitelisted seed deposit is made, and the factory neither performs nor guarantees that seed deposit. On top of this, a provider MANAGER can call setDepositWhitelistEnabled(false) before any seeding occurs, which re-opens the first-depositor inflation attack window.
	
	JavaScript
	`// deposits open with the market: require the seed or the deposit whitelist first`
	`require(IERC20(share).totalSupply() > 0 || IV3Provider(share).depositWhitelistEnabled(), V3DepositUnguarded());`
	
	Impact
	The result is that a known inflation attack vector the guard was meant to close can remain exploitable, and a market can exist in a 0-priced, non-operational state despite passing the creation check.

Recommendation	Consider only requiring totalSupply(share) > 0 at creation time, enforcing a seed-before-create ordering, which gives the strongest guarantee that the market is already seeded and immune to the first-depositor inflation attack.
Status	Acknowledged, its is presumed that deposit whitelist will not be arbitrarily enabled until a market is seeded during a V3Provider market creation.

[I02] Operational prerequisites for a fully functional V3 market are not enforced on-chain and must live in the deployment runbook	
Contract	MarketFactory.sol
Severity	Informational
Description	**Description** Several steps are required before a V3 market is fully operational, and the factory legitimately cannot perform them on-chain. These are not code gaps, but they must be

	captured in the deployment runbook so a market is not left in a half-configured state. The required post-creation steps are: 1. The seed deposit to prevent first depositor inflation attacks 2. The pool cardinality warm-up 3. Calling setPairWhitelist on V3Liquidator for the swap routers, which flashLiquidate, sellToken, and sellBNB all depend on and which the factory never wires 4. Sizing the provider/adapter risk parameters maxDailyLossUsd, maxSpotDeviationBps, maxTwapDeviationBps, and haircutBps 5. Wiring the slisBNBxMinter atomically for SlisBNBV3Provider 6. Ensuring sufficiency of the MPC cap within the SlisBNBxMinter.sol 7. Running a token/pair suitability screen to exclude unsuitable assets such as fee-on-transfer tokens, BNB-style approval tokens, tokens with hooks, pairs with a high decimal gap, and pairs with a coarse tickSpacing. 8. Additionally, V3 markets have no permissionless liquidation pathways. Generic and public liquidators are explicitly rejected from the whitelist, so liquidation liveness depends entirely on the V3Liquidator BOT. If that bot is unavailable or misconfigured, unhealthy positions cannot be liquidated by anyone else, which can let bad debt accumulate.
Recommendation	Consider documenting every one of these prerequisites as mandatory, ordered steps in the operational runbook, and verify each is completed before a market is considered live
Status	Acknowledged, it is presumed that the above operational considerations will be taken into account during each V3Provider market creation.

[I03] setV3Liquidator defers prerequisite checks and neither setter guards against a no-op	
Contract	MarketFactory.sol
Severity	Informational
Description	**Description** setV3Liquidator stores the new liquidator after confirming it is non-zero and reports the same Moolah core, but it does not verify the two conditions the creation path later enforces.

- It does not check that the liquidator's fund source is either unset or equal to the factory's liquidation vault, so an admin can store a liquidator whose fund source points at a foreign vault, after which every V3 market creation reverts until the liquidator is corrected.

- It also does not verify that the factory holds the manager role on the new liquidator, which the factory needs to configure markets, so creation reverts until that role is granted.

JavaScript

```
 /**
   * @dev Set the V3Liquidator of V3 LP share markets; it must sit on
this Moolah.
   * @param _v3Liquidator The address of the V3Liquidator contract
   */
  function setV3Liquidator(address _v3Liquidator) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_v3Liquidator != address(0), "ZeroAddress");
    require(IV3Liquidator(_v3Liquidator).MOOLAH() == address(moolah),
InvalidV3Liquidator());
    emit V3LiquidatorUpdated(address(v3Liquidator), _v3Liquidator);
    v3Liquidator = IV3Liquidator(_v3Liquidator);
  }

  /**
   * @dev Re-pin a registered share's oracle for its future markets;
existing markets keep theirs.
   * @param share The V3 share
   * @param oracle Its new V3ProviderOracle
   */
  function setV3Oracle(address share, address oracle) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(v3OracleOf[share] != address(0), InvalidV3Share());
    V3MarketLib.validate(address(moolah), share, oracle);
    v3OracleOf[share] = oracle;
    emit V3ProviderRegistered(share, oracle);
  }
```

Impact

Both failures are silent at set time, leaving the factory in a state where V3 market creation is blocked. Neither setter guards against being called with the value already stored, so re-setting the same liquidator emits a misleading update event, and re-pinning the same oracle re-validates and re-emits without any change

Recommendation	Consider the following:

	• In setV3Liquidator, when the new liquidator already has a non-zero fund source, require it to equal the factory's liquidation vault, and optionally require that the factory holds the manager role on the new liquidator, so misconfiguration is surfaced at set time rather than at the next market creation. • Add a check to both setters that rejects or skips a value equal to the current one, avoiding redundant storage writes and misleading events. If not, ensure that the DEFAULT_ADMIN_ROLE is fully trusted to not make such input errors.
Status	Acknowledged, it is presumed that the DEFAULT_AMIN_ROLE is trusted to not incur such input errors.

Notes

[N01] V3Liquidator reflow blacklist is not set for the share token, unlike the smart provider path	
Contract	MarketFactory.sol
Description	**Description** The _configSmartProvider path defensively calls setReflowBlacklist(collateral, true) on both the generic and broker liquidators, but the V3 path sets no reflow blacklist entry on the V3Liquidator. This asymmetry is currently harmless because the share token is never passed as a _reflow argument, and a share transfer initiated by a non-Moolah sender would revert anyway. **Impact** As a result this is a defense-in-depth gap rather than a live vulnerability. Still, leaving it unset means the V3 configuration is less hardened than the smart provider configuration, and it offers no protection if a future reflow path is introduced or if a custom extraLiquidators entry is wired in. **Note** Fixed, now sets the V3Provider share token within the V3Liquidator white list within _wireProvider as a defense in-depth.

[N02] V3 market creation and oracle switch has no fail fast pricing liveness check	
Contract	MarketFactory.sol
Description	**Description** When a V3 LP share market is created, the wiring validation only confirms identity relationships between the share, its adapter, and its oracle. It never confirms that the resilient oracle can actually price the two underlying legs, nor that the adapter's TWAP is ready to be read. Moolah's createMarket does call peek(collateralToken), but for a share with supply == 0 the V3 oracle short-circuits and returns 0 without ever touching the adapter, so the pricing path is not exercised at creation. **Impact** As a result, e.g. a wstETH/wbETH market, which prices the LP composition using the pool TWAP clamped to the exchange rate, can be created on a young or low cardinality pool and the creation will succeed. The first seeded peek then calls positionAmountsAt, which calls fairSqrtPriceX96, which calls pool.observe and reverts with OLD because the pool's observation history is too short for the configured TWAP period. Once that happens, every borrow, withdraw, and liquidation health check reverts, so the market is effectively bricked for all priced operations Note that none of the other market creation paths performs an explicit pricing-liveness check, but compared to common and broker markets, where the collateral oracle poke exercises the real feed at creation, the V3 path is strictly weaker because the share oracle's supply == 0 short-circuit hides a misconfigured feed or an un-warmed pool. Additionally, setV3Oracle re-pins a share's oracle after calling the shared validate() routine, but never confirms the oracle can actually produce a usable price. It does not check that the resilient oracle returns a non-zero price for each underlying leg, and it

	does not perform a trial read of the adapter's fair price. The creation path uses the same identity-only validation. As a result, an admin can re-pin, or an operator can create, a market whose oracle is correctly wired yet non-functional, for example when a leg feed is unconfigured or when the underlying V3 pool's observation window is not yet warmed and the fair-price read reverts.

[N03] createV3Market checks fundSource but not the vault to liquidator registration	
Contract	V3MarketLib.sol
Description	**Description** createV3Market requires that the V3Liquidator's fundSource() is either 0 or equal to the liquidationVault, but it never confirms that the vault actually recognizes the liquidator, that is, that liquidationVault.liquidators(v3Liq) == true. These are two separate pieces of state. **Impact** If fundSource was set during the liquidator's initialization, that path bypasses the registration step, so a liquidator can pass the fundSource check while still not being registered on the vault. In that configuration the vault funded flashLiquidate fallback, which relies on the vault's provideFund, reverts because the vault rejects the unregistered liquidator. The result is a market whose vault backed liquidation fallback is silently non operational even though creation succeeded. Note that this can be prevented with appropriate V3Liquidator registration on the LiquidationVault and/or adding an additional check in createV3Market that the liquidator is registered on the vault, requiring liquidationVault.liquidators(v3Liq) == true **Note** Fixed, now checks within createV3Market that liquidationVault.liquidators(v3Liq) == true.

[N04] setV3Liquidator and setV3Oracle do not migrate existing V3 markets	
Contract	MarketFactory.sol
Description	**Description** Both setters update only the MarketFactory's own storage, which governs markets created from that point forward. setV3Oracle changes the oracle pinned for a share's future markets, but a live market records its oracle in its Moolah market parameters at creation and Moolah exposes no function to change that value, so existing positions keep pricing against the previously pinned oracle. setV3Liquidator replaces the stored liquidator pointer, but every existing market keeps the old liquidator in its Moolah liquidation whitelist, and the old liquidator keeps its market, provider, and token whitelists. **Impact** The new liquidator is wired only into markets created after the change. The usual reason to call either setter is to retire a faulty or compromised oracle or liquidator, so an admin can reasonably believe the replacement has taken effect while every already deployed market stays bound to the old component. A faulty oracle keeps mispricing live collateral, which can produce bad debt or unfair liquidations, and a compromised

	liquidator keeps its authority over live markets. The factory offers no de-registration path and emits no signal that live-market migration is still required, so the required changes on existing v3 markets needs to be performed directly on the appropriate contracts accordingly with a different MANAGER role address. **Note** Now includes a code comment within setV3Liquidator that indicates that an update only affects future V3 markets, where existing live markets are not migrated.

[N05] MarketFactory sits near the EIP-170 bytecode limit	
Contract	MarketFactory.sol
Description	**Description** Currently, the MarketFactory runtime bytecode measures 24,132 of the 24,576-byte EIP-170 contract size limit, leaving only 444 bytes of headroom. With limited room, a future change that adds wiring to the factory risks pushing it over the limit and making it undeployable. Only one of the three market-creation families was moved out of the factory. - The V3 path was relocated into the linked external library V3MarketLib, which runs by DELEGATECALL so its body lives at the library address rather than inside the factory. - The generic path (_createMarket plus _configSmartProvider) and the fixed-term path (_createFixedTermMarket), together with the shared _whitelistToken and _wireLoanToken helpers, still live inline in the factory and account for the bulk of its remaining size. The following can be considered for future upgrades to reduce contract size limit: 1. Fold the tail of batchCreateV3Markets (the loan-token whitelist call, _wireLoanToken, the BNBProvider branch, the supply-whitelist loop, and the final event emission) into V3MarketLib.createV3Market by widening that library's Deps struct to carry WBNB, BNBProvider, revenueDistributor, buyBack, and autoBuyBack. 2. Externalize the generic path into a new CommonMarketLib and the fixed-term path into a new FixedTermMarketLib, each mirroring V3MarketLib. Once done, _whitelistToken and _wireLoanToken have no remaining factory callers and can be deleted from the factory. Keep each path in a separate library so no single library itself exceeds 24KB, keep the onlyRole and whenNotPaused guards on the thin factory wrappers, and pass the needed immutables through a Deps struct, since a library cannot read the factory immutables under DELEGATECALL. 3. Convert the remaining require statements that use string messages into custom errors. These include the ZeroAddress string repeated about 14 times, plus empty market params, array length mismatch, RateCalculator not set, BrokerLiquidator not set, StockOracleSwitch not set, and Zero broker address.

4. Remove the single-item createMarket and createFixedTermMarket external functions and have callers use the batch functions with length-1 arrays.